

Lab Report 8: Creating a Simple Car Shape

Name: _____ (Fill in your name)

Lab Program Number: 08

Date: _____ (Fill in your date)

Title

Program for Creating a Simple Car Shape Using Graphics Primitives

Objective

To design and implement a program using C graphics functions to draw a simple 2D car shape, demonstrating the use of basic graphics primitives such as lines, circles, arcs, and rectangles. This lab aims to develop skills in:

- Combining basic shapes to create complex objects
 - Understanding coordinate systems in graphics
 - Applying colors and fills to enhance visual appearance
 - Using graphics library functions effectively
-

Theory

In computer graphics, complex objects can be created using combinations of simple geometric primitives. A car shape can be constructed using:

Basic Primitives Used:

1. **Rectangles** - for the car body and top cabin
2. **Circles** - for wheels and wheel rims
3. **Lines** - for decorative details and bumpers
4. **Flood Fill** - for coloring different components

Graphics.h Library Functions:

Function	Purpose
<code>initgraph()</code>	Initialize graphics mode
<code>rectangle(x1, y1, x2, y2)</code>	Draw a rectangle
<code>circle(x, y, radius)</code>	Draw a circle
<code>line(x1, y1, x2, y2)</code>	Draw a line
<code>setcolor(color)</code>	Set drawing color
<code>setfillstyle(pattern, color)</code>	Set fill pattern and color
<code>floodfill(x, y, boundary)</code>	Fill an area with current fill style
<code>outtextxy(x, y, text)</code>	Display text at coordinates

Design Approach:

1. Plan the car structure with coordinates
2. Draw from bottom to top (wheels → body → top → details)
3. Apply appropriate colors to different components
4. Add decorative elements for realism

Code

```
c
```

```
#include <stdio.h>
#include <graphics.h>
#include <conio.h>

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, ""); // Initialize graphics mode

    // Set background color
    setbkcolor(LIGHTCYAN);
    cleardevice();

    // Car Body (main rectangle)
    setcolor(BLUE);
    setfillstyle(SOLID_FILL, BLUE);
    rectangle(100, 250, 500, 350);
    floodfill(150, 300, BLUE);

    // Car Top (upper rectangle/cabin)
    setcolor(RED);
    setfillstyle(SOLID_FILL, RED);
    rectangle(150, 200, 450, 250);
    floodfill(200, 220, RED);

    // Windows
    setcolor(CYAN);
    setfillstyle(SOLID_FILL, CYAN);
    // Left window
    rectangle(180, 210, 270, 240);
    floodfill(200, 220, CYAN);
    // Right window
    rectangle(330, 210, 420, 240);
    floodfill(350, 220, CYAN);

    // Wheels
    setcolor(BLACK);
    setfillstyle(SOLID_FILL, BLACK);
    // Left wheel
    circle(180, 360, 30);
    floodfill(180, 360, BLACK);
    // Right wheel
    circle(420, 360, 30);
    floodfill(420, 360, BLACK);
```

```
// Wheel rims (inner circles)  
setcolor(LIGHTGRAY);  
setfillstyle(SOLID_FILL, LIGHTGRAY);  
circle(180, 360, 18);  
floodfill(180, 360, LIGHTGRAY);  
circle(420, 360, 18);  
floodfill(420, 360, LIGHTGRAY);
```

```
// Headlights  
setcolor(YELLOW);  
setfillstyle(SOLID_FILL, YELLOW);  
// Front headlight  
circle(490, 290, 12);  
floodfill(490, 290, YELLOW);  
// Rear light  
setcolor(RED);  
setfillstyle(SOLID_FILL, RED);  
circle(110, 290, 12);  
floodfill(110, 290, RED);
```

```
// Door outline  
setcolor(WHITE);  
line(250, 250, 250, 340);  
line(350, 250, 350, 340);
```

```
// Decorative line on body  
setcolor(WHITE);  
line(100, 280, 500, 280);
```

```
// Bumpers  
setcolor(DARKGRAY);  
setfillstyle(SOLID_FILL, DARKGRAY);  
rectangle(85, 330, 100, 345);  
floodfill(90, 335, DARKGRAY);  
rectangle(500, 330, 515, 345);  
floodfill(505, 335, DARKGRAY);
```

```
// Window separator  
setcolor(RED);  
line(300, 210, 300, 240);
```

```
// Display text  
setcolor(BLACK);
```

```
settextstyle(DEFAULT_FONT, HORIZ_DIR, 2);
outtextxy(180, 400, "Simple Car Graphics");

// Additional details - antenna
setcolor(BLACK);
line(200, 200, 200, 180);
circle(200, 175, 5);

getch();
closegraph();
return 0;
}
```

Program Explanation

Component Breakdown:

1. Car Body (Blue Rectangle):

- Main rectangle from (100, 250) to (500, 350)
- Filled with blue color
- Forms the lower portion of the car

2. Car Top/Cabin (Red Rectangle):

- Upper rectangle from (150, 200) to (450, 250)
- Filled with red color
- Represents the passenger cabin

3. Windows (Cyan Rectangles):

- Two windows on the cabin
- Left window: (180, 210) to (270, 240)
- Right window: (330, 210) to (420, 240)
- Filled with cyan to represent glass

4. Wheels (Black Circles):

- Two circles at $y = 360$
- Left wheel at $x = 180$, radius = 30
- Right wheel at $x = 420$, radius = 30
- Inner rims (gray circles) radius = 18

5. **Lights:**

- Front headlight: Yellow circle at (490, 290)
- Rear light: Red circle at (110, 290)

6. **Details:**

- White door outlines (vertical lines)
 - Horizontal decorative line
 - Gray bumpers on both ends
 - Antenna on cabin roof
-

Output

(Attach a printed screenshot or describe the output here)

Expected Output Description:

The program displays a simple 2D car on a light cyan background with the following features:

- **Body:** Blue rectangular base (lower portion)
- **Cabin:** Red rectangular top section
- **Windows:** Two cyan rectangular windows with white separator
- **Wheels:** Two black circles with gray rims at the bottom
- **Lights:** Yellow headlight (front), red taillight (rear)
- **Details:** White door lines, gray bumpers, antenna
- **Text:** "Simple Car Graphics" displayed below the car

The car appears in a side view orientation with proper proportions and color coordination.

Key Coordinates Reference

Component	Start Point	End Point	Color
Main Body	(100, 250)	(500, 350)	Blue
Cabin	(150, 200)	(450, 250)	Red
Left Window	(180, 210)	(270, 240)	Cyan

Component	Start Point	End Point	Color
Right Window	(330, 210)	(420, 240)	Cyan
Left Wheel	(180, 360)	radius: 30	Black
Right Wheel	(420, 360)	radius: 30	Black

Conclusion

This lab successfully demonstrated how to create a simple 2D car shape using basic graphics primitives in C. The program effectively combined rectangles, circles, lines, and color fills to construct a recognizable car figure.

Key learning points:

1. **Combining Primitives:** Complex objects can be built by layering simple geometric shapes
2. **Coordinate System:** Understanding the x-y coordinate system is crucial for accurate positioning
3. **Color Management:** Using appropriate colors enhances visual clarity and realism
4. **Layering:** Drawing order matters - background elements should be drawn first
5. **Proportions:** Maintaining proper size relationships between components improves appearance

Skills Developed:

- Using graphics.h library functions effectively
- Planning object structure before coding
- Managing coordinate systems accurately
- Applying colors and fills strategically
- Creating composite graphics from basic primitives

Real-world Applications:

- Game sprite design
- Simple animation frames
- User interface icons
- Educational graphics demonstrations
- Basic CAD drawings

Possible Enhancements:

- Add animation (moving car)
- Implement user input for customization
- Add more details (mirrors, door handles)
- Create multiple vehicles
- Add background scenery (road, trees, buildings)

This exercise reinforced fundamental computer graphics concepts and showed how complex visual objects can be built from simple geometric elements, which is foundational for more advanced graphics and game development.

Instructor's Signature: _____

Date: _____

Marks: _____ / _____